Data and Algorithm Analysis

Chapter 22 — Single-Source Shortest Path

Thang Hoang

Deparment of Computer Science, Virginia Tech

Outline

Introduction

Dijkstra's Algorithm

Bellman-Ford Algorithm

Difference Constraints

Table of Contents

Introduction

Dijkstra's Algorithm

Bellman-Ford Algorithm

Difference Constraints

Definitions — Shortest Paths

Let $G = (V, E)$ be a directed graph. Let $w : E \to \mathbb{R}$ be a weight function on the edges of G. For $u, v \in V$, a **path** (not necessarily simple) from u to v is a sequence p of vertices

$$u = v_0, v_1, \ldots, v_k = v$$

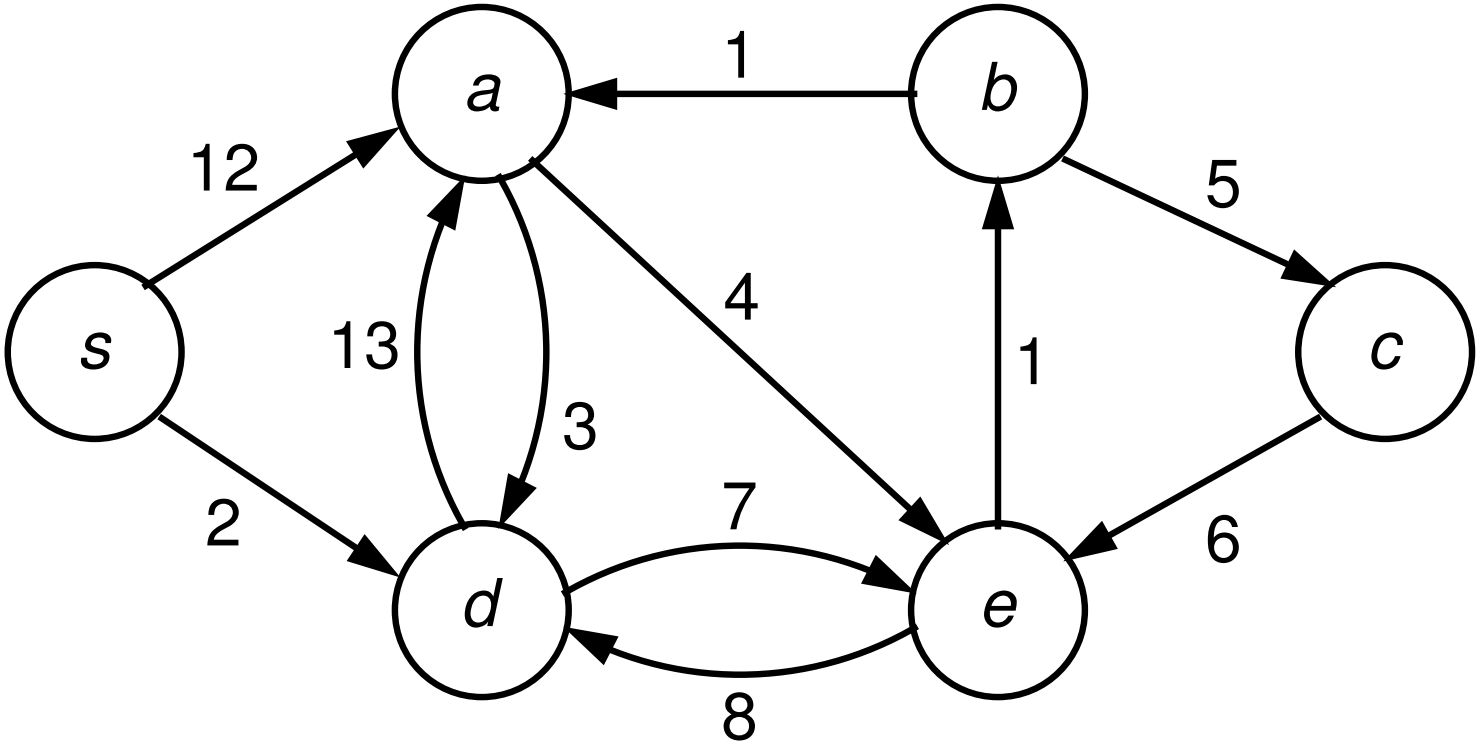
such that $(v_{i-1}, v_i) \in E$ for $1 \leq i \leq k$; we write $u \overset{p}{\rightsquigarrow} v$.
The **weight** of p is

$$w(p) \quad = \quad \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

The **shortest path weight** from u to v is

$$\delta(u, v) \quad = \quad \begin{cases} \min\{w(p) \mid u \overset{p}{\rightsquigarrow} v\} & \text{if there is } p \text{ with } u \overset{p}{\rightsquigarrow} v; \\ \infty & \text{otherwise.} \end{cases}$$

Example — Shortest Path



$\delta(s, e)$? $\delta(s, a)$?

Questions and a Problem

Question: When is $\delta(u, v) = -\infty$?

SINGLE-SOURCE SHORTEST PATH

Input: Directed graph $G = (V, E)$; edge weight function $w : E \to \mathbb{Z}$; source vertex $s \in V$.

Output: For each $v \in V$, $\delta(s, v)$.

Question: How does this differ from BFS?

Question: Applications?

Still can imagine π pointers and a shortest-path tree rooted at s.
Question: When does this not work?

Lemma 24.1

Lemma

Let $p = v_0, v_1, \ldots, v_k$ *be a shortest path from* v_0 *to* v_k*. For* $0 \leq i \leq j \leq k$*, let* $p_{ij} = v_i, v_{i+1}, \ldots, v_j$ *be a subpath. Then,* p_{ij} *is a shortest path from* v_i *to* v_j*.*

Corollary

For all edges $(u, v) \in E$*, we have* $\delta(s, v) \leq \delta(s, u) + w(u, v)$*.*

Pseudocode — Initialization

Algorithmic idea: For each $v \in V$, maintain $v.d$ as an upper bound on $\delta(s, v)$, and maintain $v.\pi$ as a pointer used to find a shortest path in reverse.

```
INITIALIZE-SINGLE-SOURCE(G, s)
    for v ∈ V
        v.d = ∞
        v.π = NIL
    s.d = 0
```

Both Dijkstra's algorithm and Bellman-Ford use this initialization.

Pseudocode — Relaxation

Both Dijkstra's algorithm and Bellman-Ford use a common step to update $v.d$ and $v.\pi$. This step is called **relaxation** of an edge (u, v).

```
RELAX(u, v, w)
    if v.d > u.d + w(u, v)
        v.d = u.d + w(u, v)
        v.π = u
```

At all times, if the π pointers lead from v back to s, then they define a path (in reverse) from s to v of length $\leq v.d$.

Table of Contents

Introduction

Dijkstra's Algorithm

Bellman-Ford Algorithm

Difference Constraints

Assumption — Dijkstra's Algorithm

Assume all edge weights are ≥ 0.

Practical assumption for many applications.

Allows the relaxation of an edge to occur only once.

Essentially a weighted version of breadth-first search.

- Instead of a FIFO queue, maintains vertices in a **priority queue** according to $v.d$ values.
- Keys are shortest-path weights ($v.d$).
- Can think of waves, like BFS.
- A wave emanates from the source.
- The first time that a wave arrives at a vertex, a new wave emanates from that vertex.

Pseudocode — Dijkstra's Algorithm

```
DIJKSTRA(G, w, s)
    INITIALIZE-SINGLE-SOURCE(G, s)
    S = ∅        // Vertices already finalized
    // Q is a min priority queue according to v.d values.
    Q = V
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        S = S ∪ {u}
        for (u, v) ∈ E
            RELAX(u, v, w)
    return v.d and v.π values
```

Looks like Prim's, but computing $v.d$, and using shortest-path weights as keys.

Look also like a greedy algorithm? Time complexity: ?

Example — Dijkstra's Algorithm

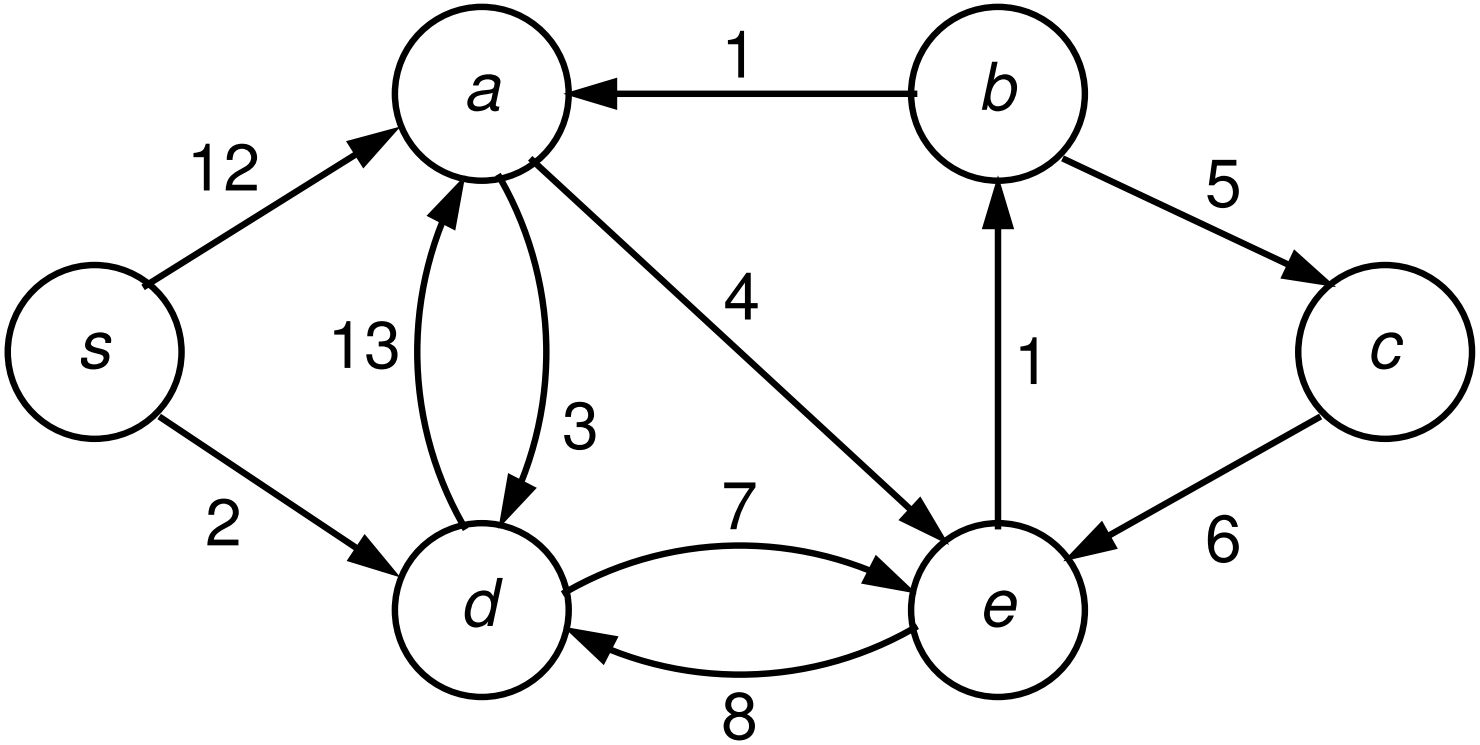


Table of Contents

Introduction

Dijkstra's Algorithm

Bellman-Ford Algorithm

Difference Constraints

Assumption — Bellman-Ford Algorithm

Allow negative edge weights.

Compute $v.d$ and $v.\pi$ for all $v \in V$

Return TRUE if no negative-weight cycles reachable from s, FALSE otherwise

Hence, all $\delta(s, v)$ values are well defined.

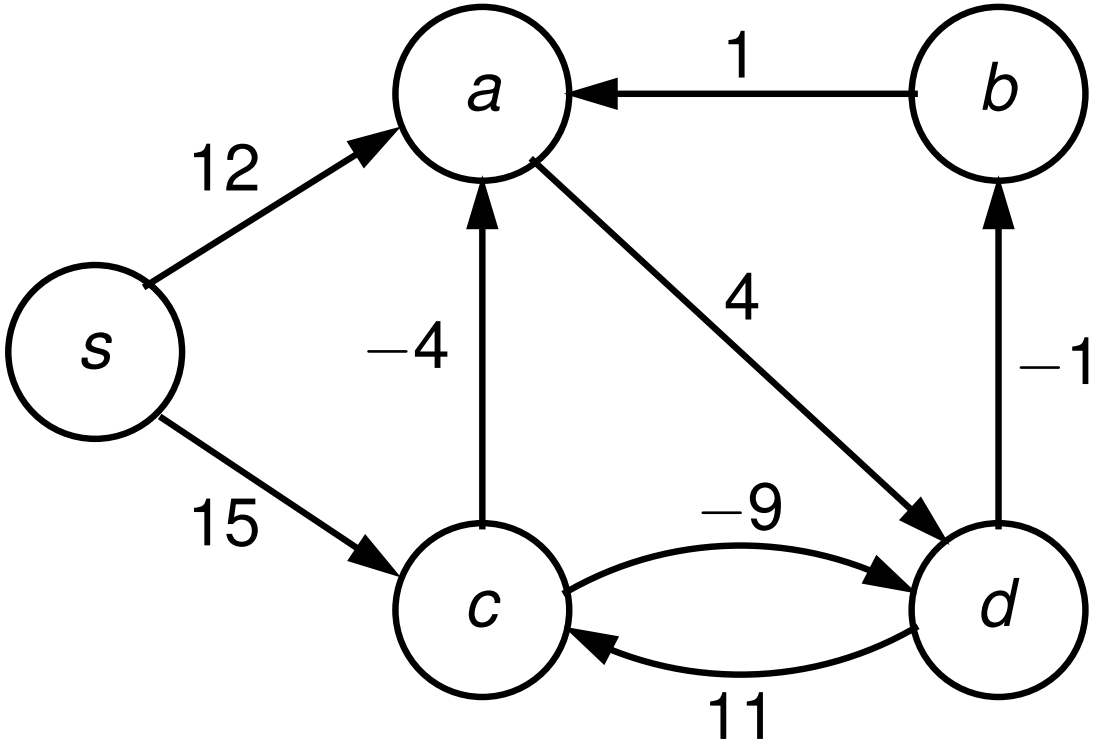
Will be able to detect that there is a negative weight cycle reachable from s.

Pseudocode — Bellman-Ford Algorithm

```
BELLMAN-FORD(G, w, s)
    // Allow negative edge weights.
    INITIALIZE-SINGLE-SOURCE(G, s)
    for i = 1 to |V| − 1
        for (u, v) ∈ E
            RELAX(u, v, w)
    for (u, v) ∈ E
        if v.d > u.d + w(u, v)
            return FALSE
    return v.d and v.π values
```

Time complexity: ?

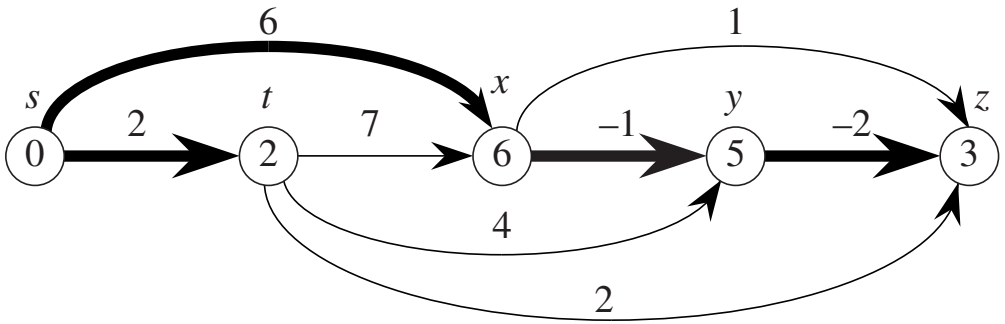
Example — Bellman-Ford Algorithm



Single-source shortest paths in a DAG

Since a DAG, there is no negative-weight cycles.

```
DAG-SHORTEST-PATHS(G, w, s)
    topologically sort the vertices of G
    INITIALIZE-SINGLE-SOURCE(G, s)
    for each vertex u ∈ G.V, taken in topologically sorted order
        for each vertex v ∈ G.adj[u]
            RELAX(u, v, w)
```



Wrap-Up — Single-Source Shortest Path

Key operation: Edge weight relaxation

When to use Dijkstra's algorithm vs. Bellman-Ford algorithm?

Dijkstra's time complexity: $\Theta((|V| + |E|) \lg |V|)$

Bellman-Ford time complexity: $\Theta(|V||E|)$

Issue: Negative-weight cycles

Table of Contents

Introduction

Dijkstra's Algorithm

Bellman-Ford Algorithm

Difference Constraints

Difference Constraints and Shortest Paths

A special case of linear programming (Chapter 29) that can reduce to finding shortest paths from a single source.

DIFFERENCE CONSTRAINTS

Input: A set of inequalities of the form $x_j - x_i \leq b_k$.

- x's are variables, $1 \leq i, j \leq n$.
- b's are variables, $1 \leq i, j \leq n$.

Output: A set of values for the x' that satisfy all m inequalities (**feasible solution**), or NONE determining no such values exist.

Example — Difference Constraints

Example

$$
\begin{aligned}
x_1 - x_2 &\leq 5 \\
x_1 - x_3 &\leq 6 \\
x_2 - x_4 &\leq 1 \\
x_3 - x_4 &\leq 2 \\
x_4 - x_1 &\leq 3
\end{aligned}
$$

Solution: $\vec{x} = (0, -4, -5, -3)$ (or $\vec{x}' = (0, -4, -5, -3) = \vec{x} + 5$)

Lemma

If x is a feasible solution, then so is $x + d$ for any constant d.

Proof?

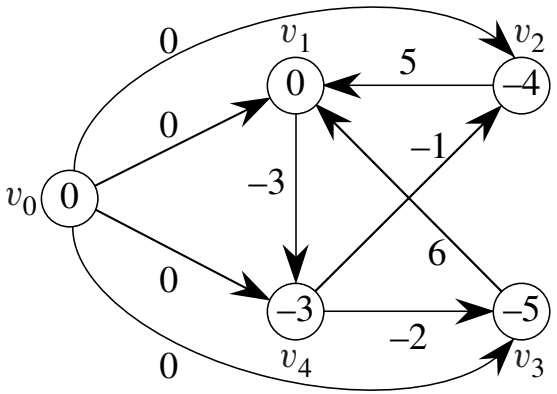
Constraint Graph

$G = (V, E)$, weighted, directed.

- $V = \{v_0, v_1, v_2, \ldots, v_n\}$: one vertex per variable $+v_0$
- $E = \{(v_i, v_j) : x_j - x_i \leq b_k$ is a constant $\} \cup \{(v_0, v_1), \ldots, (v_0, v_n)\}$
- $w(v_0, v_j) = 0$ for all $j = 1, 2, \ldots, n$
- $w(v_i, v_j) = b_k$ if $x_j - x_i \leq b_k$

Example

$$x_1 - x_2 \leq 5$$
$$x_1 - x_3 \leq 6$$
$$x_2 - x_4 \leq 1$$
$$x_3 - x_4 \leq 2$$
$$x_4 - x_1 \leq 3$$



Solving Difference Constraints

1. Form constraint graph G.
   - $n + 1$ vertices
   - $m + n$ edges
   - Complexity: $\Theta(m + n)$
2. Run BELLMAN-FORD(G, w, v_0) with source vertex v_0.
   - Complexity: $O((n + 1)(m + n)) = O(n^2 + nm)$.
3. If BELLMAN-FORD returns FALSE $\Rightarrow$ no feasible solution
4. Otherwise, set $x_i = \delta(v_0, v_i)$ for all $i = 1, 2, \ldots, n$.